



ACTIVIDAD 1

CONSTRUCCIÓN DE UNA APLICACIÓN DE TAREAS PENDIENTES (TO-DO LIST) CON ANGULAR

Asignatura: Desarrollo Web con Marcos de Trabajo en la
Interfaz Gráfica

Alumno: César Adrián Morado Rodríguez

Indice

1.	INTRODUCCIÓN	3
2.	CREACIÓN DEL PROYECTO	3
3.	CREACIÓN DEL COMPONENTE DE TAREAS	4
4.	CREACIÓN DEL MODELO DE DATOS	4
5.	CREACIÓN DEL SERVICIO DE TAREAS	5
6.	INTEGRACIÓN DEL COMPONENTE EN LA APLICACIÓN	6
7.	IMPLEMENTACIÓN DEL COMPONENTE TASKCOMPONENT	7
8.	CREACIÓN DE LA INTERFAZ DE USUARIO	8
9.	RESULTADOS OBTENIDOS	10
10.	CONCLUSIONES	10
11.	REFERENCIAS BIBLIOGRÁFICAS.....	10

1. INTRODUCCIÓN

Angular es un framework desarrollado por Google que permite crear aplicaciones web dinámicas utilizando componentes, servicios, directivas e inyección de dependencias. En esta actividad se desarrolló una aplicación de gestión de tareas (To-Do List) con el objetivo de comprender los conceptos fundamentales de Angular y aplicar buenas prácticas de desarrollo.

La aplicación permite registrar tareas, visualizar la lista de tareas existentes y eliminar aquellas que ya no sean necesarias. Para lograrlo se utilizaron componentes, servicios e interfaces.

2. CREACIÓN DEL PROYECTO

Para iniciar el desarrollo se creó una nueva aplicación Angular utilizando Angular CLI.

Comandos utilizados:

1. `ng new ToDo-App`
2. `cd ToDo-App`
3. `ng serve -o`

El comando **ng new** generó la estructura base del proyecto, incluyendo los archivos de configuración y dependencias necesarias. Posteriormente se utilizó **ng serve** para ejecutar la aplicación en un servidor local y verificar que funcionara correctamente desde el navegador.

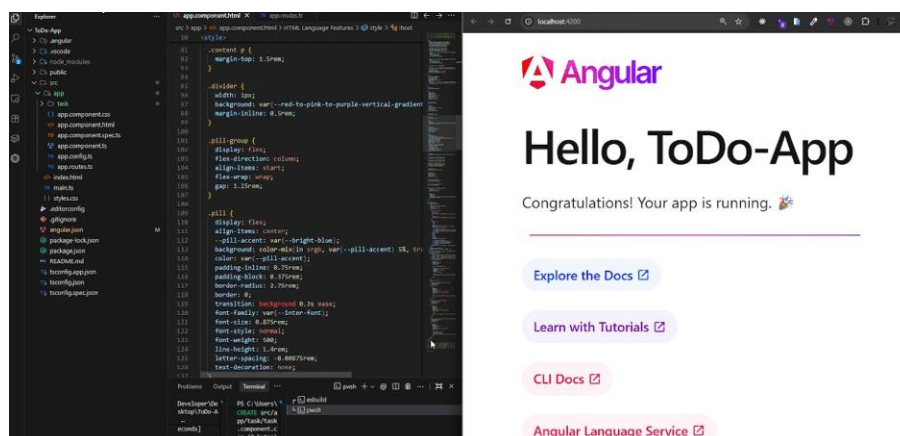


Figura 1.-Ejecución inicial de la aplicación Angular en el servidor local.

3. CREACIÓN DEL COMPONENTE DE TAREAS

Una vez creado el proyecto se generó un componente denominado TaskComponent, el cual sería responsable de mostrar la interfaz gráfica y gestionar las acciones relacionadas con las tareas.

Comando utilizado:

```
ng g c task
```

Angular generó automáticamente los siguientes archivos:

- task.component.ts
- task.component.html
- task.component.css
- task.component.spec.ts

El componente TaskComponent concentra toda la lógica relacionada con la administración de tareas dentro de la aplicación.

4. CREACIÓN DEL MODELO DE DATOS

Para definir la estructura de cada tarea se creó una interfaz denominada Task.

Código implementado:

```
export interface Task {  
  id: number;  
  title: string;  
  completed: boolean;  
}
```

La interfaz permite definir un contrato de datos para garantizar que todas las tareas tengan la misma estructura.

Descripción de los atributos:

- id: identificador único.
- title: nombre de la tarea.
- completed: indica si fue completada.

5. CREACIÓN DEL SERVICIO DE TAREAS

Con el objetivo de centralizar la lógica de negocio se creó un servicio encargado de administrar las tareas.

Comando utilizado:

- `ng g s service/task`

El servicio permite realizar operaciones sobre la colección de tareas como agregar, consultar, actualizar y eliminar registros.

Se importó la interfaz en nuestro servicio para poder hacer uso de ella.

Código implementado:

```
import { Injectable } from '@angular/core';
import { Task } from '../interface/task.interface';
@Injectable({
  providedIn: 'root'
})
export class TaskService {
  private tasks: Task[] = [];

  constructor() { }

  getTasks(): Task[] { //Obtener las tareas
    console.log(this.tasks);
    return this.tasks;
  }

  addTask(task: Task): void { //Agregar una tarea
    this.tasks.push(task);
  }

  deleteTask(id: number): void { //Eliminar una tarea
    this.tasks = this.tasks.filter(task => task.id !== id);
  }

  updateTask(task: Task): void { //Actualizar una tarea
    this.tasks = this.tasks.map(t => t.id === task.id ? task : t);
  }
}
```

El servicio permite:

- Obtener tareas.
- Agregar tareas.
- Eliminar tareas.

6. INTEGRACIÓN DEL COMPONENTE EN LA APLICACIÓN

El componente TaskComponent fue importado dentro del componente raíz AppComponent para que se mostrara automáticamente al iniciar la aplicación.

La aplicación fue desarrollada utilizando componentes Standalone de Angular. Esta característica permite importar componentes directamente mediante la propiedad imports sin necesidad de utilizar AppModule, simplificando la estructura del proyecto y siguiendo las prácticas recomendadas en versiones recientes de Angular.

Código implementado:

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';
import { TaskComponent } from '../task/task.component';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, TaskComponent],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'ToDo-App';
}
```

Dentro del archivo app.component.html se agregó el selector correspondiente:

```
<app-task> </app-task>
<router-outlet />
```

De esta manera Angular renderiza el componente de tareas al iniciar la aplicación.

7. IMPLEMENTACIÓN DEL COMPONENTE TASKCOMPONENT

El componente TaskComponent utiliza el servicio TaskService para interactuar con la lista de tareas.

```
import { Component } from '@angular/core';
import { TaskService } from '../service/task.service';
import { Task } from '../task';
import { FormsModule } from '@angular/forms';
import { CommonModule } from '@angular/common';

@Component({
  selector: 'app-task',
  imports: [CommonModule, FormsModule],
  templateUrl: './task.component.html',
  styleUrls: ['./task.component.css']
})
export class TaskComponent {
  task: Task[] = [];
  taskNewTitle: string = "";
  constructor(private taskService: TaskService) { }

  ngOnInit(): void {
    this.task = this.taskService.getTasks();
  }

  addTask(): void {
    const newTask: Task = {
      id: this.task.length + 1,
      title: this.taskNewTitle,
      completed: false
    }
    this.taskService.addTask(newTask);
    console.log("Agregando una nueva tarea")
  }

  deleteTask(id: number): void {
    this.taskService.deleteTask(id);
    this.task = this.taskService.getTasks();
  }
}
```

Se importó FormsModule para utilizar la directiva ngModel y realizar el enlace bidireccional de datos entre la vista y el componente. Asimismo, se importó CommonModule para utilizar directivas estructurales como *ngFor y *ngIf.

Explicación:

- Se utiliza inyección de dependencias para acceder al servicio.
- El método `ngOnInit()` carga las tareas al iniciar el componente.
- El método `addTask()` agrega nuevas tareas.
- El método `deleteTask()` elimina tareas existentes.

8. CREACIÓN DE LA INTERFAZ DE USUARIO

Para la interfaz gráfica se utilizó HTML junto con Bootstrap.

Características implementadas:

- Campo de texto para capturar nuevas tareas.
- Botón para agregar tareas.
- Lista dinámica de tareas.
- Botón para eliminar tareas.
- Mensaje cuando no existen tareas registradas.

Conceptos de Angular utilizados:

- `[(ngModel)]` para enlazar el contenido del input.
- `*ngFor` para recorrer la lista de tareas.
- `*ngIf` para mostrar mensajes condicionales.
- `(click)` para capturar eventos de botones.
- `(ngSubmit)` para procesar formularios.

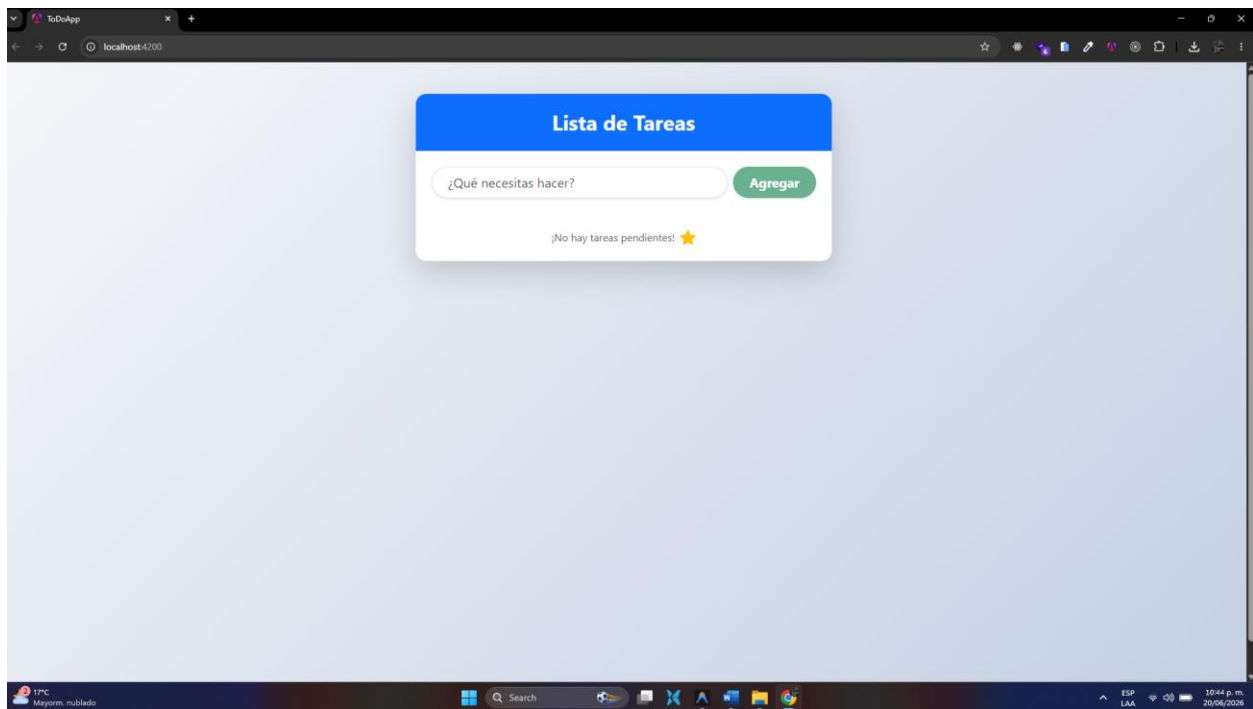


Figura 2.- Aplicación sin tareas registradas

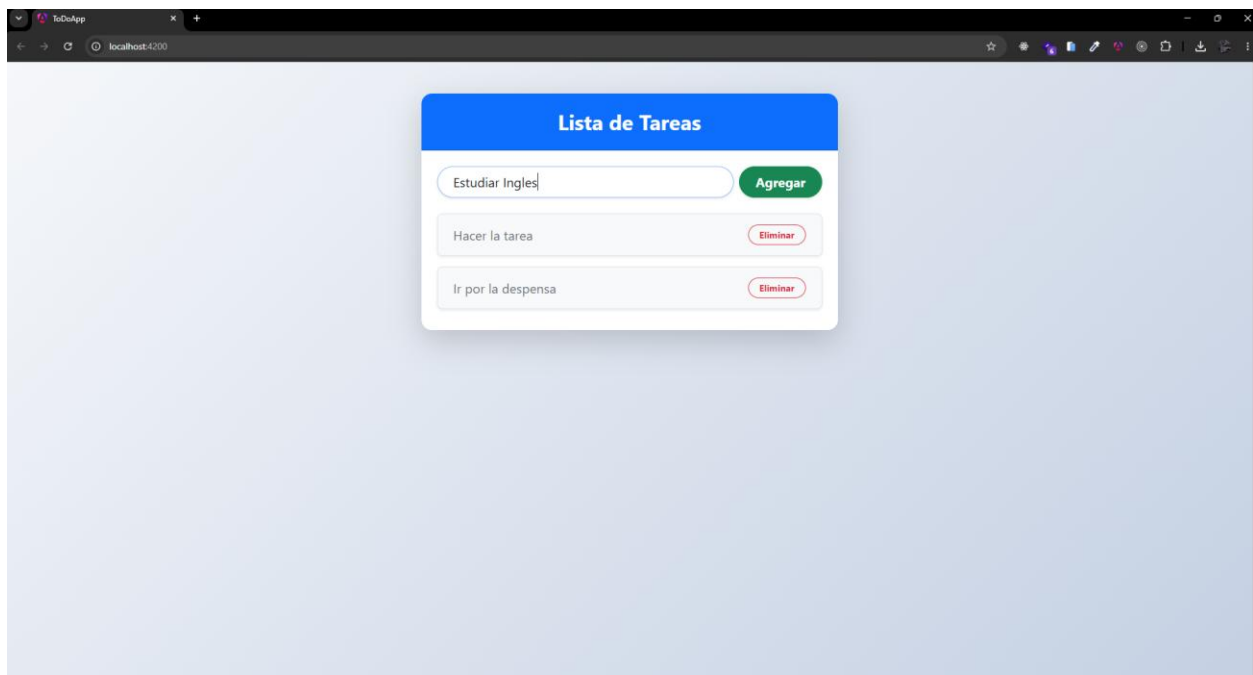


Figura 3. Aplicación con tareas agregadas dinámicamente mediante TaskService.

9. RESULTADOS OBTENIDOS

La aplicación desarrollada cumple con los objetivos planteados al permitir:

- Registrar nuevas tareas.
- Visualizar tareas registradas.
- Eliminar tareas existentes.
- Gestionar información mediante servicios.
- Aplicar Data Binding y directivas estructurales.

Durante las pruebas realizadas se verificó el correcto funcionamiento de cada una de las operaciones implementadas.

10. CONCLUSIONES

El desarrollo de esta actividad permitió comprender la arquitectura básica de Angular mediante la utilización de componentes, servicios e interfaces. Asimismo, se aplicaron conceptos fundamentales como la inyección de dependencias, las directivas estructurales y el enlace de datos.

La implementación de una aplicación To-Do List resultó adecuada para comprender la comunicación entre componentes y servicios, así como la organización del código dentro de una aplicación Angular moderna.

Links de la aplicación To-Do-App en Angular:

<https://to-do-app-h53d2.ondigitalocean.app/>

Github:

<https://github.com/moradoadrian/To-Do-App>

11. REFERENCIAS BIBLIOGRÁFICAS

Angular. (2025). Angular Documentation. Recuperado de <https://angular.dev>

Microsoft. (2025). TypeScript Documentation. Recuperado de <https://www.typescriptlang.org>

Node.js Foundation. (2025). Node.js Documentation. Recuperado de <https://nodejs.org>

Bootstrap Team. (2025). Bootstrap Documentation. Recuperado de <https://getbootstrap.com>